Module 5 Week B — Reading: Trees, Ensembles & Model Selection

In Week A you built a complete ML evaluation pipeline: train/test splitting with stratification, cross-validation, classification and regression metrics, and linear models with regularization. You now have the tools to train a model, evaluate it honestly, and avoid the most common mistakes.

But linear models assume that the relationship between features and the target can be captured by a weighted sum. That assumption works well for many problems — and fails quietly on many others. When the decision boundary is nonlinear, when features interact in ways that a straight line cannot represent, linear models underfit systematically. This week introduces a fundamentally different approach: tree-based models that partition the feature space recursively rather than projecting it onto a line.

The arc of this reading: trees as a response to linear limitations, ensembles as a response to tree instability, class imbalance as a reality check on evaluation, calibrated probabilities as the bridge between raw predictions and actionable decisions, and experiment tracking as professional practice. By the end of this week, you will compare every model you have built across both weeks and write a decision memo recommending one for deployment.

What success looks like: By the end of Week B, you can build and evaluate decision tree and random forest models, handle class imbalance with class weights and threshold tuning, interpret calibrated probabilities, produce a model comparison table with a decision memo tied to operational KPIs, and structure reproducible ML workflows with experiment logging and model persistence.

In This Reading

1. Why Trees?
2. Decision Trees
3. Random Forests
4. Class Imbalance
5. Precision-Recall Curves and PR-AUC
6. Calibrated Probabilities
7. Reproducible ML Workflows
8. Lightweight Experiment Tracking
9. Model Comparison as a Decision Process

- Key Terms
- Summary
- Further Reading
- What's Next

---

1. Why Trees?

The Linear Boundary Problem

Logistic regression draws a single straight line (or hyperplane, in higher dimensions) to separate classes. When the true decision boundary is linear or close to it, this works well. But consider a customer churn dataset where customers with very low tenure churn, customers with very high tenure also churn (they are exploring competitors), and customers in the middle stay. The relationship between tenure and churn is not monotonic — no single line captures it.

A **decision tree** handles this naturally. Instead of fitting a line, it asks a series of yes/no questions: "Is tenure less than 6 months? If yes, predict churn. If no, is tenure greater than 48 months? If yes, predict churn. Otherwise, predict no churn." The result is a set of rectangular regions in the feature space, each assigned a prediction. The boundaries are axis-aligned — always parallel to a feature axis — but by combining many such splits, a tree can approximate complex, nonlinear boundaries.

When Trees Outperform Linear Models

Trees tend to outperform linear models when:

- The relationship between features and the target is nonlinear
- Features interact (e.g., high income combined with low credit utilization predicts something different from either feature alone)
- The data contains categorical features with many levels (trees handle these without one-hot encoding overhead)
- The decision boundary is irregular or discontinuous

Trees tend to underperform linear models when:

- The true relationship is approximately linear
- The dataset is very small (trees need enough data to split meaningfully)
- High-dimensional, sparse data (text features, for example)

The key insight: trees and linear models make different assumptions about the structure of the data. Neither is universally better. Knowing when to use each is a core skill.

---

2. Decision Trees

How a Tree Grows

A **decision tree** is a supervised learning model that makes predictions by recursively splitting the data into subsets based on feature values. Each internal **node** in the tree represents a test on a single feature (e.g., "Is `monthly_charges` > 50?"). Each branch represents the outcome of that test. Each terminal **leaf** node holds a prediction — a class label for classification, or a mean value for regression.

The algorithm works top-down:

1. Start with all training examples at the root node.
2. For every feature and every possible split point, compute a measure of how well the split separates the target classes.
3. Choose the feature and split point that produces the best separation.
4. Create two child nodes: one for examples where the condition is true, one where it is false.
5. Repeat recursively for each child node until a stopping condition is met.

The stopping condition is critical — without one, the tree keeps splitting until every leaf contains a single training example, achieving 100% training accuracy by memorizing the data.

Splitting Criteria: Gini Impurity and Information Gain

The "best separation" at each node is measured by how pure the resulting child nodes are. A node is **pure** if all its examples belong to the same class.

Gini impurity measures how often a randomly chosen element from the node would be misclassified if it were randomly labeled according to the distribution of classes in that node:

Gini(node) = 1 - sum(p_i^2) for each class i

where p_i is the proportion of class i in the node. A pure node has Gini = 0. For binary classification, a node with two equally represented classes has Gini = 0.5 — the maximum for a binary split. (More generally, with k equally represented classes the maximum Gini is 1 − 1/k, so ≈0.667 for three classes and 0.75 for four. The binary case is what you will work with in this module.) The algorithm picks the split that results in the greatest weighted reduction in Gini impurity across the two child nodes.

Information gain uses **entropy** instead of Gini:

Entropy(node) = -sum(p_i * log2(p_i)) for each class i

Information gain is the reduction in entropy from a split. In practice, Gini impurity and information gain produce very similar trees. scikit-learn uses Gini by default.

Complexity Controls

A decision tree with no constraints will grow until every leaf is pure — perfectly fitting the training data and almost certainly overfitting. The key hyperparameters that control complexity are:

- **`max_depth`** : The maximum number of levels in the tree. A tree with `max_depth=5` makes at most 5 sequential decisions. Lower values produce simpler, more generalizable models. This is the single most important regularization control for decision trees.
- **`min_samples_split`** : The minimum number of samples required to split a node. If a node has fewer than this many samples, it becomes a leaf. Default is 2 (effectively no constraint). Setting this to 10 or 20 prevents the tree from creating splits on tiny groups.
- **`min_samples_leaf`** : The minimum number of samples in a leaf node. Even if a split would reduce impurity, it is not allowed if either child would have fewer than `min_samples_leaf` samples.

Pruning is the process of removing branches that do not improve generalization. scikit-learn supports cost-complexity pruning via `ccp_alpha` , but in practice, setting `max_depth` is the most common and effective approach.

Edge case: `max_depth=None` (the default) grows trees to full depth. On small data sets, this almost always overfits. Always set `max_depth` explicitly when training a standalone decision tree.

Training and Visualizing a Decision Tree

```python
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Assume X_train, X_test, y_train, y_test from a prior split
dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X_train, y_train)

# Visualize the first 3 levels
fig, ax = plt.subplots(figsize=(20, 10))
plot_tree(dt, max_depth=3, feature_names=X_train.columns,
          class_names=["No Churn", "Churn"], filled=True, ax=ax)
plt.tight_layout()
plt.savefig("decision_tree.png", dpi=150)
plt.show()
```

The `filled=True` parameter colors nodes by their majority class, with darker colors indicating higher purity. This visualization is one of the major advantages of decision trees: you can inspect exactly how the model makes decisions.

scikit-learn provides both `DecisionTreeClassifier` (for classification) and `DecisionTreeRegressor` (for regression, where the splitting criterion minimizes squared error and leaves predict mean values).

---

3. Random Forests

The Variance Problem

A single decision tree is **high variance**: small changes in the training data can produce a very different tree. Train on a slightly different sample, and the first split might use a different feature, cascading into an entirely different structure. This brittleness is the fundamental weakness of decision trees.

Bagging: Many Trees, One Vote

A **random forest** is an **ensemble** of many decision trees, each trained independently, whose predictions are combined by majority vote (classification) or averaging (regression). The key idea is **bagging** (bootstrap aggregating):

1. From the original training set of N examples, draw N examples **with replacement** — this is a **bootstrap sample**. Some examples appear multiple times; others are left out entirely. On average, each bootstrap sample contains about 63% of the unique training examples.
2. Train a full-depth decision tree on this bootstrap sample.
3. Repeat steps 1-2 for `n_estimators` trees (commonly 100-500).
4. To predict, pass the input through every tree and take the majority vote.

Each tree is individually overfit (full depth, no pruning), but because each tree sees a different bootstrap sample, they overfit in different ways. Averaging their predictions cancels out the individual noise — reducing **variance** while maintaining the low-bias benefits of deep trees.

Feature Randomness

Bagging alone helps, but if one feature is overwhelmingly predictive, every tree will split on it first, and the trees will be highly correlated. Correlated trees provide less variance reduction when averaged.

Random forests add a second source of randomness: at each split, instead of considering all features, the algorithm randomly selects a subset of `max_features` features and finds the best split among only those. The default for classification is `max_features='sqrt'` — the square root of the total number of features. This forces each tree to use different features near the top, **decorrelating** the trees and improving the ensemble's variance reduction.

Key Hyperparameters

- **`n_estimators`** : Number of trees. More trees generally improve performance up to a point, after which returns diminish. 100 is a reasonable default; 200-500 for more demanding problems. **Edge case:** With very few estimators (e.g., `n_estimators=10`), the forest may underperform a single well-tuned deep tree — there are not enough trees for the variance reduction to take effect.
- **`max_features`** : Number of features considered at each split. `'sqrt'` (default for classification) or `'log2'` are standard. Lower values increase tree diversity (more decorrelation) but increase individual tree bias.
- **`max_depth`** : Can be set to limit tree depth, but random forests often work well with `max_depth=None` (full-depth trees) because the ensemble averaging mitigates the overfitting of individual trees.

Out-of-Bag Score

Each bootstrap sample leaves out about 37% of the training examples. These **out-of-bag (OOB)** examples were not used to train that particular tree, so they serve as a natural validation set. The **OOB score** is computed by predicting each training example using only the trees that did not include it in their bootstrap sample. This gives a free estimate of generalization performance without needing a separate validation set.

```python
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100, random_state=42, oob_score=True
)
rf.fit(X_train, y_train)
print(f"OOB Score: {rf.oob_score_:.3f}")
```

The OOB score is a convenience, not a replacement for proper cross-validation. Use it as a quick sanity check during development.

Feature Importance

Random forests provide a built-in measure of feature importance via `feature_importances_` . This is the **mean decrease in impurity (MDI)**: for each feature, it sums up how much that feature's splits reduced Gini impurity (or entropy) across all trees, normalized to sum to 1.0.

```python
import pandas as pd

importances = pd.Series(
    rf.feature_importances_, index=X_train.columns
).sort_values(ascending=False)

# Plot top 10
importances.head(10).plot(kind="barh")
plt.xlabel("Feature Importance (MDI)")
plt.title("Top 10 Features — Random Forest")
plt.tight_layout()
plt.savefig("feature_importances.png")
plt.show()
```

Edge case: MDI-based feature importance is biased toward **high-cardinality features** — features with many unique values (like a continuous variable or an ID-like column) get more opportunities to split and appear more important than they truly are. For a more reliable measure, use **permutation importance** (available via `sklearn.inspection.permutation_importance`), which measures how much the model's score drops when a feature's values are randomly shuffled. This is covered in the challenge extensions.

---

4. Class Imbalance

Why Accuracy Misleads

You encountered accuracy as a metric in Week A. Now consider a concrete scenario: in Petra Telecom's dataset, only about 15% of customers actually churned. A model that predicts "no churn" for every single customer achieves 85% accuracy — without learning anything. The model has zero **recall** (it catches none of the churners) and undefined **precision** (it never predicts churn).

Accuracy is misleading on **class-imbalanced** data because it is dominated by the majority class. A dataset where 95% of examples are negative rewards any model that mostly predicts negative.

This is not just an academic concern. In fraud detection, disease screening, equipment failure prediction, and customer churn — the domains where ML is most valuable — the positive class is almost always rare. If your evaluation metric does not account for this, you will deploy models that miss the events you most need to catch.

Strategy 1: Class Weights

scikit-learn allows you to set `class_weight='balanced'` in most classifiers. This adjusts the loss function so that misclassifying a minority-class example incurs a higher penalty, proportional to the class's scarcity:

weight_i = n_samples / (n_classes * n_samples_in_class_i)

For a dataset with 85% negative and 15% positive, the positive class gets a weight roughly 5.7 times higher than the negative class. The model is penalized much more for missing a positive example.

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Without class weighting
rf_default = RandomForestClassifier(n_estimators=100, random_state=42)
rf_default.fit(X_train, y_train)
print("Default weights:")
print(classification_report(y_test, rf_default.predict(X_test)))

# With class weighting
rf_balanced = RandomForestClassifier(
    n_estimators=100, class_weight='balanced', random_state=42
)
rf_balanced.fit(X_train, y_train)
print("Balanced weights:")
print(classification_report(y_test, rf_balanced.predict(X_test)))
```

Edge case: `class_weight='balanced'` is not a free fix. It typically improves recall (catches more positives) but reduces precision (more false positives). In a churn context, this means you will correctly identify more churners but also incorrectly flag more loyal customers. Whether this tradeoff is acceptable depends on the business context — the cost of missing a churner versus the cost of annoying a loyal customer with an unnecessary retention offer.

Strategy 2: Threshold Tuning

By default, scikit-learn classifiers predict the positive class when `predict_proba` returns a probability above 0.5. But 0.5 is arbitrary — there is no reason it should be the optimal cutoff for your problem.

Threshold tuning means choosing a custom **decision threshold**. Instead of calling `model.predict(X)` , you call `model.predict_proba(X)` and apply your own threshold:

```python
import numpy as np

# Get predicted probabilities for the positive class
y_proba = rf_balanced.predict_proba(X_test)[:, 1]

# Apply a custom threshold
threshold = 0.3
y_pred_custom = (y_proba >= threshold).astype(int)

print(f"Threshold: {threshold}")
print(classification_report(y_test, y_pred_custom))
```

Lowering the threshold below 0.5 makes the model more aggressive — it predicts positive more often, increasing recall but decreasing precision. Raising the threshold makes the model more conservative — it only predicts positive when very confident, increasing precision but decreasing recall.

Threshold tuning gives you explicit control over the precision-recall tradeoff without retraining the model. You train once, then choose the threshold that matches your operational requirements.

Strategy 3: SMOTE (Awareness Only)

> ***For Awareness Only.*** *SMOTE and related synthetic oversampling techniques are not required in this module's assignments. Focus your implementation time on `class_weight='balanced'` and threshold tuning, which are the assessed strategies. The brief coverage below exists so you recognize the technique when you encounter it in ML literature and industry contexts — not so you apply it here.*

SMOTE (Synthetic Minority Over-sampling Technique, in the `imbalanced-learn` library / `imblearn`) generates synthetic minority-class examples by interpolating between existing ones rather than duplicating them, creating new points along the line segments connecting nearest neighbors in the minority class. It can help when class weights alone aren't enough, but has practical limits — it only works cleanly in continuous feature spaces, and can create unrealistic examples when the minority class has multiple disjoint clusters. For this module's assignments, `class_weight='balanced'` and threshold tuning are sufficient.

---

5. Precision-Recall Curves and PR-AUC

The Precision-Recall Curve

A **precision-recall curve** plots precision (y-axis) against recall (x-axis) at every possible decision threshold. As you lower the threshold from 1.0 to 0.0, recall increases (you predict positive more often, catching more true positives) and precision typically decreases (more of your positive predictions are wrong).

The ideal model has a curve that hugs the top-right corner — high precision at high recall. A random classifier on imbalanced data produces a flat line at y = (positive class proportion).

```python
from sklearn.metrics import PrecisionRecallDisplay

fig, ax = plt.subplots(figsize=(8, 6))

PrecisionRecallDisplay.from_estimator(
    rf_default, X_test, y_test, name="RF (default)", ax=ax
)
PrecisionRecallDisplay.from_estimator(
    rf_balanced, X_test, y_test, name="RF (balanced)", ax=ax
)

ax.set_title("Precision-Recall Curves")
plt.tight_layout()
plt.savefig("pr_curves.png")
plt.show()
```

PR-AUC and Average Precision

PR-AUC is the area under the precision-recall curve. It summarizes the model's performance across all thresholds into a single number between 0 and 1. Higher is better. scikit-learn computes this as **average precision** (`average_precision_score`), which is a weighted mean of precisions at each threshold, weighted by the increase in recall:

```python
from sklearn.metrics import average_precision_score

y_proba_default = rf_default.predict_proba(X_test)[:, 1]
y_proba_balanced = rf_balanced.predict_proba(X_test)[:, 1]

ap_default = average_precision_score(y_test, y_proba_default)
ap_balanced = average_precision_score(y_test, y_proba_balanced)

print(f"PR-AUC (default): {ap_default:.3f}")
print(f"PR-AUC (balanced): {ap_balanced:.3f}")
```

Why PR-AUC Over ROC-AUC for Imbalanced Data

You may have encountered **ROC-AUC** (area under the receiver operating characteristic curve). ROC-AUC plots the true positive rate against the false positive rate. The problem: on imbalanced data, the false positive rate can be very low even when the model produces many false positives in absolute terms, because it is divided by the large number of true negatives. ROC-AUC can make a mediocre model look excellent when negatives vastly outnumber positives.

PR-AUC does not use true negatives at all — it focuses entirely on how well the model handles the positive class. For problems where the positive class is rare and important (churn, fraud, disease, failure), PR-AUC gives a more honest picture of model quality.

Rule of thumb: Use ROC-AUC when classes are roughly balanced. Use PR-AUC when the positive class is a small minority and is the class you care about. A common convention is "~20%" as the dividing line, but this is a heuristic rather than a hard boundary — if your positive class is 18% or 22%, the same reasoning applies. The real question is whether true negatives dominate the denominator of your metrics, and that depends on the specific problem.

---

6. Calibrated Probabilities

What predict_proba Actually Returns

When you call `model.predict_proba(X)` , the returned values are not true probabilities in the statistical sense. For a random forest, the "probability" for a sample is the fraction of trees that voted for each class. For logistic regression, it is derived from the sigmoid function applied to the linear combination. These outputs are scores that rank examples by confidence, but they may not correspond to actual event frequencies.

A model's predicted probability of 0.8 should mean: "of all the examples I assign a probability of 0.8, about 80% actually belong to the positive class." When this correspondence holds, the model is **well-calibrated**.

Why Calibration Matters

Calibration is critical whenever you act on probability values rather than just on predicted labels. Consider Petra Telecom's churn prediction system: "Alert the retention team if a customer's churn probability exceeds 0.7." If the model says 0.7 but the true positive rate at that score is only 0.35, half the alerts are wasted. Business decisions that depend on thresholds — alerts, offers, escalation protocols — require calibrated probabilities.

Reliability Diagrams

A **reliability diagram** (also called a calibration curve or calibration plot) bins predictions by their predicted probability and plots the mean predicted probability (x-axis) against the actual fraction of positives in each bin (y-axis). A perfectly calibrated model follows the diagonal. Points above the diagonal mean the model is under-confident (reality is higher than the prediction); points below mean the model is over-confident.

```python
from sklearn.calibration import CalibrationDisplay

fig, ax = plt.subplots(figsize=(8, 6))

CalibrationDisplay.from_estimator(
    rf_default, X_test, y_test, n_bins=10,
    name="RF (default)", ax=ax
)
CalibrationDisplay.from_estimator(
    rf_balanced, X_test, y_test, n_bins=10,
    name="RF (balanced)", ax=ax
)

ax.set_title("Calibration Curves")
plt.tight_layout()
plt.savefig("calibration.png")
plt.show()
```

Edge case: `predict_proba` from tree-based models is often poorly calibrated. Because trees output hard votes and the forest averages them, the probabilities tend to cluster near 0 and 1 with few values in between. This is a known limitation of random forests.

CalibratedClassifierCV

`CalibratedClassifierCV` wraps a classifier and fits a calibration mapping (Platt scaling or isotonic regression) using cross-validation:

```python
from sklearn.calibration import CalibratedClassifierCV
from sklearn.ensemble import RandomForestClassifier

# Pass an UNFITTED estimator. With cv=5, CalibratedClassifierCV will train
# the base estimator and the calibration mapping together using internal
# cross-validation — any prior .fit() call on the base estimator is
# ignored. (If you want to reuse an already-fitted model instead, use
# cv='prefit' and fit the wrapper on a separate held-out calibration set.)
base_rf = RandomForestClassifier(
    n_estimators=200, max_depth=10, class_weight='balanced',
    random_state=42, n_jobs=-1
)
calibrated_rf = CalibratedClassifierCV(base_rf, cv=5, method='sigmoid')
calibrated_rf.fit(X_train, y_train)

# Now predict_proba returns better-calibrated probabilities
y_proba_calibrated = calibrated_rf.predict_proba(X_test)[:, 1]
```

The `method='sigmoid'` (Platt scaling) fits a logistic regression to map raw scores to probabilities — works well when the calibration curve is roughly sigmoid-shaped. The `method='isotonic'` fits a non-parametric monotonic function — more flexible but prone to overfitting with small data sets. Use sigmoid as the default; try isotonic if you have enough data and the calibration curve is clearly non-sigmoid.

---

7. Reproducible ML Workflows

Why Reproducibility Matters

If you run the same script twice and get different results, you cannot trust either result. If a colleague cannot reproduce your evaluation, your comparison table is meaningless. Reproducibility is not a nice-to-have in ML — it is a requirement for any result you plan to act on.

Script Structure

Notebooks are for exploration. Scripts are for production and reproducible evaluation. A well-structured ML experiment script follows a predictable layout:

```python
"""
Model comparison experiment — Petra Telecom churn prediction.
Produces comparison table, PR curves, calibration plots, and experiment log.
"""

# --- Configuration ---
RANDOM_SEED = 42
N_FOLDS = 5
DATA_PATH = "data/telecom_churn.csv"
OUTPUT_DIR = "results"

# --- Imports ---
import os
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold
# ... more imports

# --- Data Loading ---
def load_and_preprocess(path):
    """Load data and return X, y."""
    # ...

# --- Model Definitions ---
def get_models():
    """Return dict of model_name: model."""
    # ...

# --- Evaluation ---
def evaluate_models(X, y, models, n_folds, seed):
    """Cross-validate all models and return results DataFrame."""
    # ...

# --- Output ---
def save_results(results, output_dir):
    """Save comparison table, plots, model, and log."""
    # ...

# --- Main ---
if __name__ == "__main__":
    X, y = load_and_preprocess(DATA_PATH)
    models = get_models()
    results = evaluate_models(X, y, models, N_FOLDS, RANDOM_SEED)
    save_results(results, OUTPUT_DIR)
```

This structure makes it obvious where to find each component and makes it possible to re-run the experiment by executing a single command: `python model_comparison.py` .

Seed Management

Randomness appears in multiple places in an ML pipeline: train/test splitting, cross-validation fold assignment, model initialization (random forests, neural networks), and data shuffling. Set `random_state` everywhere:

```python
# In train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# In cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# In model initialization
rf = RandomForestClassifier(n_estimators=100, random_state=42)
```

Use the same seed value throughout a single experiment. Document it in the configuration section. This ensures that running the script twice produces identical results.

Model Persistence with joblib

Once you have selected a model, save it so it can be loaded later without retraining:

```python
from joblib import dump, load

# Save the trained model
dump(rf_balanced, "results/best_model.joblib")

# Load it back (in another script or session)
loaded_model = load("results/best_model.joblib")
predictions = loaded_model.predict(X_new)
```

`joblib` is included with scikit-learn — no separate installation needed. Use `from joblib import dump, load` (not `import joblib`). The saved file contains the entire fitted model object, including learned parameters, hyperparameters, and preprocessing state if saved within a pipeline.

---

8. Lightweight Experiment Tracking

The Experiment Log

Every time you train and evaluate a model, record the results. The simplest approach is a CSV file with one row per experiment:

```python
import csv
from datetime import datetime

def log_experiment(log_path, model_name, hyperparams, metrics):
    """Append one row to the experiment log CSV."""
    row = {
        "model_name": model_name,
        "hyperparams": str(hyperparams),
        "accuracy": metrics["accuracy"],
        "precision": metrics["precision"],
        "recall": metrics["recall"],
        "f1": metrics["f1"],
        "pr_auc": metrics["pr_auc"],
        "timestamp": datetime.now().isoformat()
    }

    file_exists = os.path.exists(log_path)
    with open(log_path, "a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=row.keys())
        if not file_exists:
            writer.writeheader()
        writer.writerow(row)
```

Note: The log appends one row per call. If you rerun the same experiment (same model name and hyperparameters), the log accumulates duplicates. When reading the log to compare runs, filter by `timestamp` or deduplicate explicitly if you need unique entries.

Reading and Comparing Runs

The experiment log is a plain CSV — read it with pandas to compare runs:

```python
log = pd.read_csv("results/experiment_log.csv")
print(log.sort_values("f1", ascending=False).to_string(index=False))
```

This gives you a complete history of every model you tried, what hyperparameters you used, and how it performed. When someone asks "why did you choose this model?" you can point to the log.

Professional Tools

In production ML teams, experiment tracking is handled by tools like **MLflow** and **Weights & Biases**. These provide dashboards, artifact storage, model versioning, and team collaboration features. The principle is identical to what you are building with a CSV: record every experiment so that results are reproducible and comparable. The CSV approach teaches the pattern; the tools scale it to teams and production systems. You are not required to use these tools in this course, but knowing they exist prepares you for professional ML work.

---

9. Model Comparison as a Decision Process

Beyond "Best Accuracy Wins"

You now have models from two families: linear models (Week A) and tree-based models (Week B). Your comparison table might show that the balanced random forest has the best F1 score, but the logistic regression has better precision, and the default random forest has the best accuracy. Which is "best"?

The answer depends on the operational context, not the metric table alone.

The Decision Memo

A **decision memo** is a short written recommendation (2-3 paragraphs) that translates your model comparison into a business recommendation. It addresses:

1. **Which model and why.** Name the recommended model and cite the specific metrics that support the choice. "The balanced random forest achieves the highest recall (0.72) and the best PR-AUC (0.55), making it the strongest model for identifying churners."

2. **What tradeoffs were made.** Every model choice involves tradeoffs. Be explicit: "This model's precision (0.45) is lower than the logistic regression's (0.62), meaning approximately half of the flagged customers will not actually churn. The retention team will need to account for this false positive rate in their outreach capacity planning."

3. **How this connects to operational KPIs.** Metrics are meaningless without business context. For Petra Telecom, relevant KPIs might include: customer retention rate (recall determines how many churners are caught early), false alarm cost (precision determines how many retention offers go to non-churners), and early warning lead time (calibrated probabilities determine when the system triggers alerts). The memo should map at least one model metric to a specific business outcome.

4. **One limitation or risk.** No model is perfect. Acknowledge what could go wrong: "The model assumes that churn patterns remain stable over time. If Petra Telecom changes its pricing structure or a competitor enters the market, the model's predictions will degrade — concept drift should be monitored quarterly."

Factors Beyond Metrics

When comparing models for real deployment, metrics are necessary but not sufficient. Consider:

- **Interpretability:** Logistic regression coefficients are directly interpretable ("a 1-unit increase in monthly charges increases the log-odds of churn by 0.3"). Random forests are harder to explain — feature importance gives a rough picture, but individual predictions require tools like SHAP (SHapley Additive exPlanations) — see Further Reading below. If the model's predictions must be explained to regulators or customers, interpretability matters.
- **Latency:** A single logistic regression prediction is a dot product — microseconds. A random forest with 500 trees requires 500 tree traversals — still fast, but slower. For real-time systems processing thousands of predictions per second, this difference matters.
- **Deployment complexity:** A logistic regression model is a small set of coefficients — easy to deploy, easy to version, easy to debug. A random forest is a large collection of trees — larger file size, more complex serialization. Simpler models are easier to maintain.
- **Fairness:** Does the model perform equally well across customer segments? If it catches churn accurately for high-value customers but misses churn among lower-value customers, there is a fairness concern even if overall metrics look good.

Structured Comparison Template

Your comparison table should include all models evaluated, with rows for each metric and columns for mean and standard deviation from cross-validation:

Model	Accuracy	Precision	Recall	F1	PR-AUC
DummyClassifier (baseline)	0.85 +/- 0.01	0.00 +/- 0.00	0.00 +/- 0.00	0.00 +/- 0.00	0.15 +/- 0.01
LogisticRegression (L2, C=1.0)	…	…	…	…	…
…	…	…	…	…	…

The DummyClassifier baseline is essential — it establishes the floor. Any model that does not meaningfully outperform the dummy on recall and F1 is not useful.

Key Terms

Decision tree – A supervised learning model that predicts by recursively partitioning the feature space through a series of binary splits on individual features.

Node – A point in the tree where a split decision is made (internal node) or a prediction is stored (leaf node).

Leaf – A terminal node in a decision tree that contains the final prediction for examples that reach it.

Split – A binary decision at a node that divides examples into two groups based on a feature threshold.

Depth – The number of edges from the root node to the deepest leaf. Controls model complexity.

Pruning – The process of removing tree branches that do not improve generalization performance.

Gini impurity – A measure of node impurity: 1 minus the sum of squared class proportions. A pure node has Gini = 0.

Information gain – The reduction in entropy (or impurity) achieved by a split. The algorithm picks the split with the highest information gain.

Entropy – A measure of disorder in a node: the negative sum of p_i * log2(p_i) for each class i.

Random forest – An ensemble of decision trees, each trained on a bootstrap sample with random feature subsets, combined by majority vote or averaging.

Bagging – Bootstrap aggregating: training multiple models on bootstrap samples and combining their predictions to reduce variance.

Bootstrap sample – A sample of N examples drawn with replacement from a dataset of N examples. On average contains about 63% of the unique original examples.

Ensemble – A collection of multiple models whose predictions are combined (by voting, averaging, or other methods) to produce a final prediction.

Feature importance – A score indicating how much each feature contributes to the model's predictions. In random forests, computed as the mean decrease in impurity (MDI) across all trees.

Out-of-bag (OOB) score – A validation estimate for random forests computed using the ~37% of training examples left out of each tree's bootstrap sample.

Class imbalance – A dataset condition where one class has significantly fewer examples than another, causing standard metrics like accuracy to be misleading.

Class weight – A parameter that adjusts the loss function to penalize misclassification of underrepresented classes more heavily. Set via `class_weight='balanced'` in scikit-learn.

Threshold tuning – Choosing a custom decision threshold for `predict_proba` instead of using the default 0.5, to control the precision-recall tradeoff.

SMOTE – Synthetic Minority Over-sampling Technique: generates synthetic minority-class examples by interpolating between existing minority-class neighbors.

Precision-recall curve – A plot of precision versus recall at every possible decision threshold, summarizing a model's performance on the positive class.

PR-AUC – Area under the precision-recall curve. Summarizes positive-class prediction quality across all thresholds. More informative than ROC-AUC for imbalanced problems.

Average precision – A weighted mean of precisions at each threshold, equivalent to PR-AUC. Computed by `average_precision_score` in scikit-learn.

Calibrated probability – A predicted probability that corresponds to the actual frequency of the positive class. A calibrated score of 0.8 means ~80% of examples with that score are truly positive.

Reliability diagram – A calibration plot showing predicted probabilities (x-axis) against observed positive rates (y-axis). A well-calibrated model follows the diagonal.

predict_proba – A scikit-learn method that returns class probability estimates for each example, used for threshold tuning and calibration evaluation.

Decision threshold – The probability cutoff above which a model predicts the positive class. Default is 0.5; can be tuned to optimize for precision or recall.

Reproducibility – The ability to rerun an experiment and obtain identical results. Requires fixed random seeds, deterministic code, and documented procedures.

Seed management – Setting `random_state` parameters consistently across all random operations (splitting, model training, cross-validation) to ensure reproducibility.

Experiment log – A structured record (typically CSV) of every model trained, including hyperparameters, metrics, and timestamps, enabling systematic comparison.

Model persistence – Saving a trained model to disk so it can be loaded and used later without retraining. Typically done with `joblib.dump` / `joblib.load` .

joblib – A Python library (bundled with scikit-learn) for efficiently serializing Python objects, commonly used for saving and loading trained ML models.

Summary

Week B introduced a fundamentally different model family — decision trees and random forests — and the evaluation tools needed when real-world complications arise. Decision trees partition the feature space recursively, handling nonlinear relationships that linear models miss, but they are prone to overfitting. Random forests combine many trees through bagging and feature randomness to reduce variance while preserving the ability to capture complex patterns.

Class imbalance exposes the limitations of accuracy as a metric. You learned two assessed strategies: class weights adjust the loss function, and threshold tuning gives you explicit control over precision versus recall. (SMOTE, a synthetic oversampling technique, was covered for awareness but is not required in this module's assignments.) PR-AUC provides a single-number summary of positive-class prediction quality that is more honest than ROC-AUC on imbalanced data.

Calibrated probabilities bridge the gap between model outputs and actionable decisions — a prediction of 0.8 only means something if 80% of examples at that score are truly positive. Reliability diagrams and `CalibratedClassifierCV` let you verify and improve calibration.

Finally, reproducible workflows — structured scripts, seed management, experiment logging, model persistence — turn exploratory analysis into reliable, auditable engineering. The model comparison table and decision memo bring it all together: not "highest accuracy wins" but "which model best serves the operational context."

Further Reading

Topics referenced in this reading that you may want to explore beyond the course:

- **SHAP (SHapley Additive exPlanations)** — for individual-prediction interpretability on tree-based models. The `shap` library is the standard implementation; SHAP explains each prediction as a sum of contributions from each feature, which is especially useful when predictions must be justified to stakeholders, regulators, or customers.
- **MLflow** and **Weights & Biases** — professional experiment tracking at team scale. Dashboards, artifact storage, model versioning, and team collaboration features. The CSV-based experiment log you built in Section 8 teaches the pattern; these tools scale it to production ML teams.
- **`imbalanced-learn`** **(imblearn)** — SMOTE and related sampling techniques for when class weights and threshold tuning aren't sufficient. Useful in industry contexts with severely imbalanced data (fraud detection, rare-disease prediction).

What's Next

Module 6 moves from structured, tabular data to unstructured text. You will learn how Natural Language Processing (NLP) converts raw text into representations that ML models can work with — tokenization, text preprocessing, vectorization, and text classification. The modeling skills from Module 5 carry forward directly: you will evaluate NLP models with the same metrics, cross-validation, and comparison workflows you built here. The difference is the data: text requires its own preprocessing pipeline before the familiar sklearn workflow applies.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.